

ComposeRAG: A Modular and Composable RAG for Corpus-Grounded Multi-Hop Question Answering

Ruofan Wu*

University of Houston
rwu13@cougarnet.uh.edu

Youngwon Lee

Seoul National University
ywlee@ldi.snu.ac.kr

Fan Shu

University of Houston
fshu@cougarnet.uh.edu

Danmei Xu

Snowflake AI Research
danmei.xu@snowflake.com

Seung-won Hwang

Seoul National University
seungwonh@snu.ac.kr

Zhewei Yao[†]

Snowflake AI Research
zhewei.yao@snowflake.com

Yuxiong He

Snowflake AI Research
yuxiong.he@snowflake.com

Feng Yan

University of Houston
fyan5@central.uh.edu

Abstract

Retrieval-Augmented Generation (RAG) systems are increasingly diverse, yet many suffer from monolithic designs that tightly couple core functions like query reformulation, retrieval, reasoning, and verification. This limits their interpretability, systematic evaluation, and targeted improvement, especially for complex multi-hop question answering. We introduce **ComposeRAG**, a novel modular abstraction that decomposes RAG pipelines into atomic, composable modules. Each module, such as Question Decomposition, Query Rewriting, Retrieval Decision, and Answer Verification, acts as a parameterized transformation on structured inputs/outputs, allowing independent implementation, upgrade, and analysis. To enhance robustness against errors in multi-step reasoning, ComposeRAG incorporates a self-reflection mechanism that iteratively revisits and refines earlier steps upon verification failure. Evaluated on four challenging multi-hop QA benchmarks, ComposeRAG consistently outperforms strong baselines in both accuracy and grounding fidelity. Specifically, it achieves up to a 15% accuracy improvement over fine-tuning-based methods and up to a 5% gain over reasoning-specialized pipelines under identical retrieval conditions. Crucially, ComposeRAG significantly enhances grounding: its verification-first design reduces ungrounded answers by over 10% in low-quality retrieval settings, and by approximately 3% even with strong corpora. Comprehensive ablation studies validate the modular architecture, demonstrating distinct and additive contributions from each component. These findings underscore ComposeRAG’s capacity to deliver flexible, transparent, scalable, and high-performing multi-hop reasoning with improved grounding and interpretability.

1 Introduction

Large Language Models (LLMs), including prominent examples like GPT-3, GPT-4, and LLaMA [4, 1, 24], have demonstrated remarkable proficiency in various NLP tasks. Nevertheless, their

*Work done during an internship with Snowflake AI Research.

[†]Project Lead. The code is released at Arctic Agentic RAG.

reliance on static, pre-trained knowledge renders them susceptible to generating inaccuracies (hallucinations) [12] and limits their access to up-to-date or domain-specific information [9]. Retrieval-Augmented Generation (RAG) [17] has emerged as a critical paradigm to overcome these deficiencies by integrating external knowledge sources, thereby enhancing factual grounding. While foundational RAG approaches (e.g., RETRO [3], RE-RAG [16]) have advanced single-step QA, they encounter significant challenges when confronted with complex multi-hop queries. Such queries necessitate not only information retrieval but also sophisticated processes of decomposition and step-by-step reasoning across multiple documents—a capability that remains a substantial hurdle for conventional RAG frameworks.

The intricacies of multi-hop QA have spurred the development of specialized pipeline strategies. Initial attempts, including chain-of-thought prompting [28] and dedicated systems like MDR [30] and TTQA-RS [2], aimed to structure this complex reasoning. However, these early solutions often exhibited rigidity and were susceptible to error propagation from initial steps. While subsequent systems such as RQ-RAG [5] and Search-o1 [18] introduced greater flexibility, they also presented new challenges, including dependencies on supervised fine-tuning or inadequate verification mechanisms that result in poorly supported answers. A common deficiency across many existing multi-hop QA systems is their tendency towards monolithic or opaque architectures, which fundamentally limits their interpretability, adaptability, and potential for systematic enhancement.

In this work, we propose a modular and composable multi-hop RAG pipeline, **ComposeRAG**, that emphasizes grounding, transparency, and flexibility. Our framework consists of independently interchangeable modules—such as question decomposition, passage reranking, query rewriting, retrieval decision, and answer verification—coordinated through prompt-based interfaces. To enhance robustness in complex scenarios, we incorporate a self-reflection mechanism that enables the system to iteratively revise its reasoning steps based on past outputs. Unlike prior systems, our approach explicitly enumerates and modularizes each component of the multi-hop reasoning process. This enables fine-grained analysis, targeted upgrades, and flexible adaptation to new tasks or domains.

We evaluate ComposeRAG on four multi-hop QA benchmarks—HotpotQA, 2WikiMultiHopQA, MuSiQue, and Bamboogle—and compare against strong baselines including RQ-RAG and Search-o1 across multiple LLM backbones. Our framework consistently outperforms these baselines in both accuracy and grounding fidelity, achieving up to a 15% improvement in overall accuracy compared to fine-tuning-based methods and a 5% gain over reasoning-specialized pipelines under identical retrieval conditions. It generates more faithfully grounded answers, enables efficient early exits, and scales effectively across model sizes. Notably, ComposeRAG’s verification-first design reduces ungrounded answers by over 10% in low-quality retrieval settings, and by approximately 3% even when high-quality corpora are used. Comprehensive ablation studies confirm the value of modularity, revealing distinct and additive contributions from each module. Together, these results demonstrate ComposeRAG’s ability to deliver flexible, interpretable, and high-performing multi-hop reasoning with improved factual grounding.

Our key contributions include:

- **Modular Architecture:** A flexible and interpretable design for Multi-Hop QA that supports plug-and-play reasoning components, allowing for transparent analysis and targeted improvements (Section 4).
- **Orchestration with Self-Reflection:** An iterative orchestration strategy from self-reflecting question decomposition and other reasoning steps to recover from early errors and enhance overall robustness, as validated by performance gains with increased reflection steps (Section 6.1).
- **Well-composed:** With these contributions, we report empirical studies that confirm ComposeRAG is well-composed by:
 - Quantifying the distinct and additive performance contributions of core modules such as Question Decomposition, Passage Reranking, and Answer Verification (Section 6.2).
 - Confirming independent module upgradability, where enhancing individual components (e.g., by leveraging more powerful LLMs like GPT-4o over GPT-4o-mini for specific tasks) in isolation yields measurable improvements, showcasing the system’s scalability and adaptability (Section 6.3).

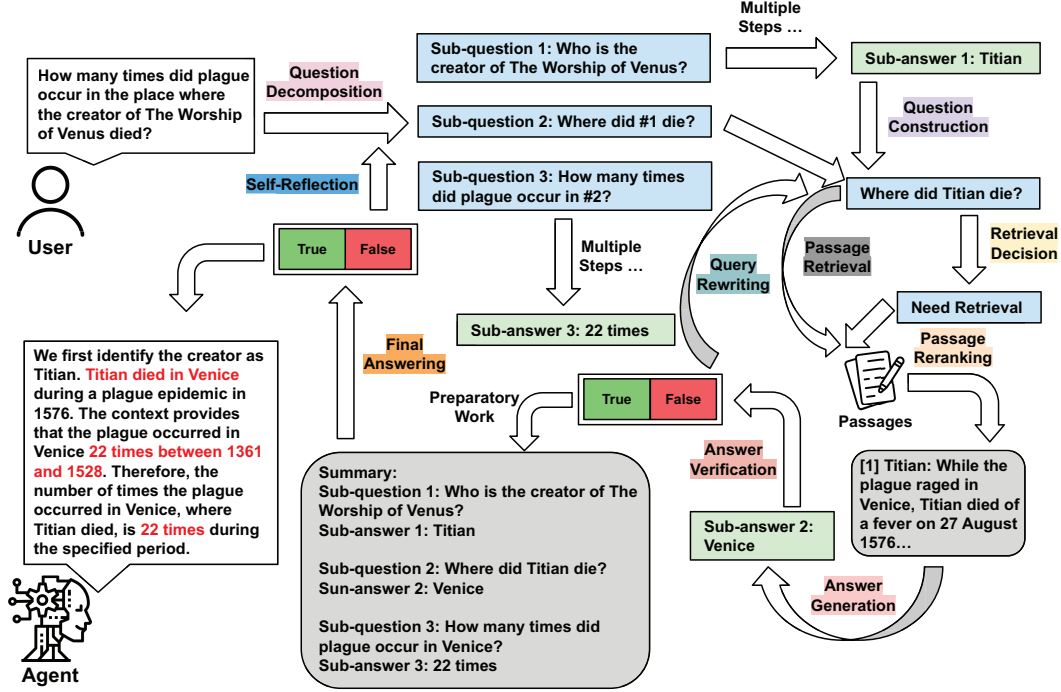


Figure 1: An overview of our modular Multi-Hop QA pipeline, highlighting the use of specialized modules to solve complex questions via step-by-step reasoning.

- Underscoring the positive impact of efficiency-oriented components like the Retrieval Decision module and the Simple QA pipeline on performance and resource utilization (Section 6.4).

2 Related Work

The development of ComposeRAG builds upon several key areas of research in question answering and information retrieval.

RAG for Open-Domain QA: LLMs for open-domain QA often leads to issues with factual consistency and the recall of specific or infrequently encountered information. RAG [17] was introduced to mitigate these limitations by enabling LLMs to access and condition their generation on information retrieved from external corpora. For instance, Atlas [11] demonstrated improved few-shot learning by combining a dense retriever with a sequence-to-sequence generator. Despite these advancements, RAG systems employing a single retrieval pass often struggle with complex questions that require synthesizing information from multiple documents through step-by-step reasoning.

Multi-Hop QA: Addressing the demands of multi-hop questions, which require reasoning across multiple pieces of evidence, has led to specialized techniques focusing on structured reasoning and query decomposition. Methods like GenDec [29] generate independent, self-contained sub-questions that can be answered individually, thereby reducing the risk of error propagation inherent in sequential multi-step reasoning. Concurrently, significant effort has been directed towards enhancing the quality of retrieved information. RankGPT [22], for example, utilizes the zero-shot capabilities of LLMs to re-rank retrieved passages based on instructional prompts, prioritizing more relevant documents. Similarly, SuRe [15] improves pipeline robustness by summarizing retrieved passages for each candidate answer, assessing the level of support, and ultimately selecting the most well-grounded response. These approaches highlight the dual importance of effective decomposition and high-quality evidence in tackling multi-hop QA.

RAG Reasoning Methods: Closest line of work to ours is recent RAG systems, tightly integrating reasoning capabilities. RQ-RAG [5] fine-tunes a query rewriting module to improve multi-hop

retrieval quality, enabling more accurate downstream generation but requiring task-specific training. In contrast, Search-o1 [18] adopts an agentic approach, prompting large reasoning models to iteratively query search engines and synthesize answers. ReARTeR [23] introduces a trust-based reward model that reinforces trustworthy intermediate reasoning steps. R1-Searcher [21] applies reinforcement learning in two stages to train LLMs to decide when and how to search effectively. Search-R1 [13] further integrates multi-turn search and reasoning through reinforcement learning with outcome-based rewards and retrieved token masking.

Our Distinction: We propose a composable RAG reasoning framework, ComposeRAG, where each module positively contributes to performance and can be independently upgraded for further gains. Modules are orchestrated by **self-reflection**, to iteratively reassess, refine, and optimize retrieval and reasoning processes. We will further distinguish our framework in Section 3.2.

A notable example of self-reflection is self-ask prompting [20], where LLMs are guided to generate and answer intermediate sub-questions before arriving at a final answer, fostering more coherent and transparent reasoning. EfficientRAG [36] builds on this by introducing an iterative retrieval strategy that generates follow-up queries without repeated LLM invocations, thus minimizing computational costs while filtering irrelevant content. AUTO-RAG [33] extends this concept by framing retrieval as a multi-turn dialogue, enabling the model to adaptively refine its queries and determine when sufficient information has been gathered. Unlike methods that focus on query refinement or sub-question planning, we aim to integrate self-reflection with the full reasoning pipeline.

3 Motivations

While recent systems such as RQ-RAG [5] and Search-o1 [18] have advanced the state of multi-hop question answering (QA), they contend with critical limitations that hinder their broader applicability and robustness:

First, **Rigidity in Component Management and Upgradability.** Existing systems, such as RQ-RAG, often employ monolithic designs where core modules are tightly coupled. This structural rigidity makes it difficult to independently substitute, upgrade, or analyze individual components. Consequently, experimental agility is reduced, and adapting the pipeline to new tasks or leveraging improved individual models becomes a substantial re-engineering effort. This underscores the need for a truly modular architecture that supports independent component enhancement, a key aspect of ComposeRAG’s design and validated by our ablation studies (Section 6.3).

Second, **Deficits in Modularity and Reasoning Transparency.** While some systems like Search-o1 demonstrate strong multi-step reasoning capabilities, their architectures may not explicitly isolate or formalize the distinct stages of the reasoning process. This lack of explicit modularity can obscure internal decision-making, making it challenging to pinpoint sources of error, attribute them to specific functional blocks, or systematically diagnose reasoning flaws. As a result, system interpretability is diminished, especially when analyzing failures. This highlights the importance of a transparent, modular framework like ComposeRAG, where each step is an identifiable and analyzable component (see Section 4.1 and Section 4.2 for design and Section 6.2 for validation).

Third, **Insufficient Grounding and Lack of Robust Verification Loops.** Advanced reasoning pipelines can sometimes produce plausible-sounding answers that lack adequate grounding in the retrieved evidence, as observed in systems like Search-o1. The absence of rigorous verification mechanisms at each step, or the inability to iteratively refine reasoning based on verification failures, elevates the risk of factual hallucinations and undermines trustworthiness. This points to the necessity for systems that not only verify answers but can also engage in self-correction—a core feature of ComposeRAG through its Answer Verification module and Self-Reflection Mechanism (see Section 4.1 and Section 4.3 for design and Section 6.2 for validation).

These limitations collectively highlight the need for a Multi-Hop QA framework that is inherently modular, transparent, and robust. Such a framework should empower researchers and practitioners with the flexibility to manage and upgrade components independently, offer clear insights into the reasoning process for effective debugging and improvement, ensure strong factual grounding through systematic verification, and incorporate mechanisms for iterative self-correction. ComposeRAG is designed to embody these principles.

4 ComposeRAG

Addressing the limitations of monolithic and opaque RAG systems outlined in Section 3, we introduce **ComposeRAG**. This framework embodies a modular and composable approach tailored for the complexities of multi-hop question answering. The foundational principle of ComposeRAG is the decomposition of the intricate multi-hop reasoning process into a series of atomic, interchangeable modules. Each module is conceptualized as a parameterized transformation, operating on structured inputs to produce structured outputs. This design facilitates independent implementation, rigorous analysis, and targeted upgrades of individual components.

To ensure meaningful modularity, we adopt the formal notion of a *well-composed system*, where each module M_i contributes measurably to system performance. Specifically, we assess:

- **Ablation:** Removing M_i yields a drop in performance P : $\Delta P_{-i} = P(S) - P(S \setminus M_i) \geq 0$.
- **Upgrade:** Substituting M_i with a better version M'_i improves performance: $\Delta P_{+i} = P(S[M_i \leftarrow M'_i]) - P(S) > 0$.

These two properties, denoted as **P1** and **P2** guide our study in Section 6, validating both properties hold in ComposeRAG, in Section 6.2 and Section 6.3, respectively: Well-composed system offers granular control by isolating the functionality of each module, and allows for precise interventions when errors occur.

4.1 Composable Building Blocks: The Core Modules of ComposeRAG

The ComposeRAG framework is constructed from a suite of specialized, yet composable, modules, each engineered to perform a distinct function within the overarching question answering process. These modules are systematically categorized based on their primary contribution to the reasoning pipeline, encompassing decomposition, evidence handling, and verified answering.

4.1.1 Decomposition and Stepwise Reasoning

Central to tackling multi-hop questions is the ability to deconstruct a complex question into a sequence of manageable sub-questions. The **Question Decomposition** module is responsible for this initial transformation, linearly breaking down the input question into simpler, interrelated sub-questions. Unlike approaches that enforce rigid tree-like structures, ComposeRAG employs a flexible placeholder-based annotation system to signify dependencies between these sub-questions. Subsequently, the **Question Construction** module takes these decomposed, placeholder-annotated sub-questions and instantiates them into actionable queries by filling in the placeholders with answers derived from preceding reasoning steps. This ensures that each sub-question presented to the retrieval and answering components is self-contained and contextually complete, as illustrated in the example (Fig. 2).

4.1.2 Evidence Retrieval and Prioritization

Effective reasoning is critically dependent on the retrieval and judicious prioritization of high-quality, relevant evidence. To this end, ComposeRAG incorporates several modules. The **Retrieval Decision** module acts as an efficiency-enhancing gatekeeper, assessing whether the current (sub-)question already contains sufficient information for an answer or if external document retrieval is indeed necessary (see Fig. 3 for an illustration).

When retrieval is deemed necessary, the **Query Rewriting** module can be invoked to enhance retrieval precision, particularly for under-specified or context-dependent sub-questions. It reformulates such queries into more complete and effective search terms, leveraging information from prior answers and the broader conversational context. Once a set of potentially relevant passages is retrieved, the **Passage Reranking** module, inspired by recent work such as [22], reorders these passages based on their specific relevance to the current sub-question. This is achieved using prompt-based scoring mechanisms to elevate the most pertinent information for subsequent processing.

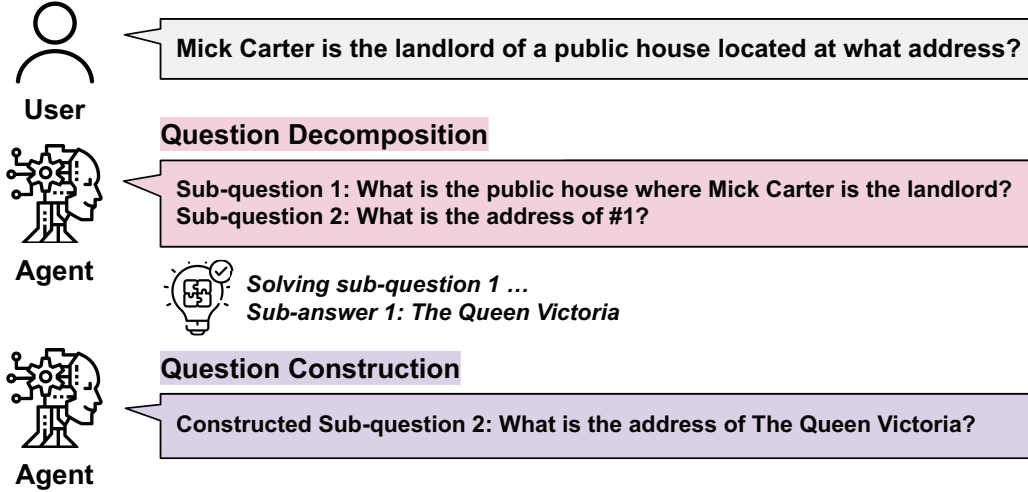


Figure 2: Question Decomposition and Construction Example

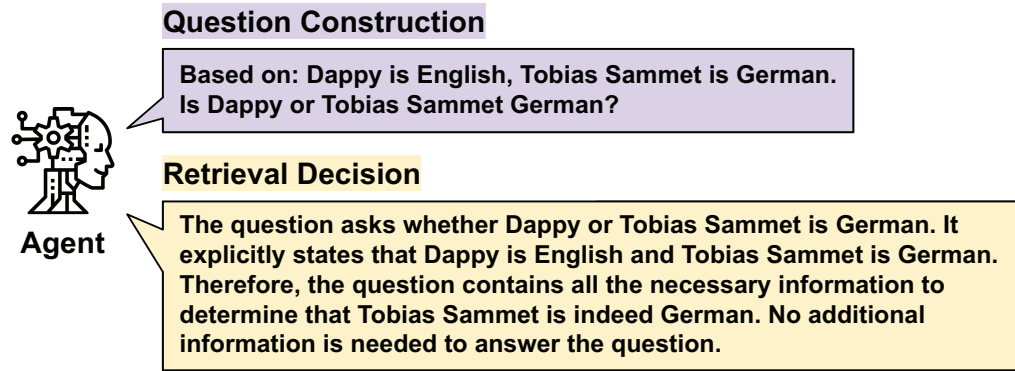


Figure 3: Retrieval Decision Example

4.1.3 Answering with Evidence-Based Verification

Ultimately, the goal is to generate faithful, well-grounded answers. The **Answer Generation** module synthesizes a candidate answer for the current sub-question, utilizing the prioritized retrieved passages and, in multi-hop scenarios, the accumulated history of prior sub-questions and their answers. A core tenet is to produce answers that are explicitly supported by the provided evidence. To ensure this, the **Answer Verification** module critically evaluates the generated answer for factual consistency and adequate grounding against the cited passages. If the supporting evidence is deemed insufficient, or if the answer is found to be ungrounded, this module can trigger corrective actions within the pipeline, such as re-attempting a previous step, or signal an abstention if a high-confidence, verifiable answer cannot be produced. Finally, for multi-hop questions, once all constituent sub-questions have been successfully processed and their answers verified, the **Final Answering** module aggregates these intermediate results to synthesize a comprehensive and coherent final answer to the original complex query. Formal definitions and detailed operational parameters for each of these modules are provided in Appendix A.1.

4.2 Orchestrating Reasoning: The ComposeRAG Pipelines

The core modules, as defined in Section 4.1, serve as the fundamental building blocks for two distinct, yet interconnected, question answering pipelines within the ComposeRAG framework: the Simple QA Pipeline and the Advanced Multi-Hop QA Pipeline.

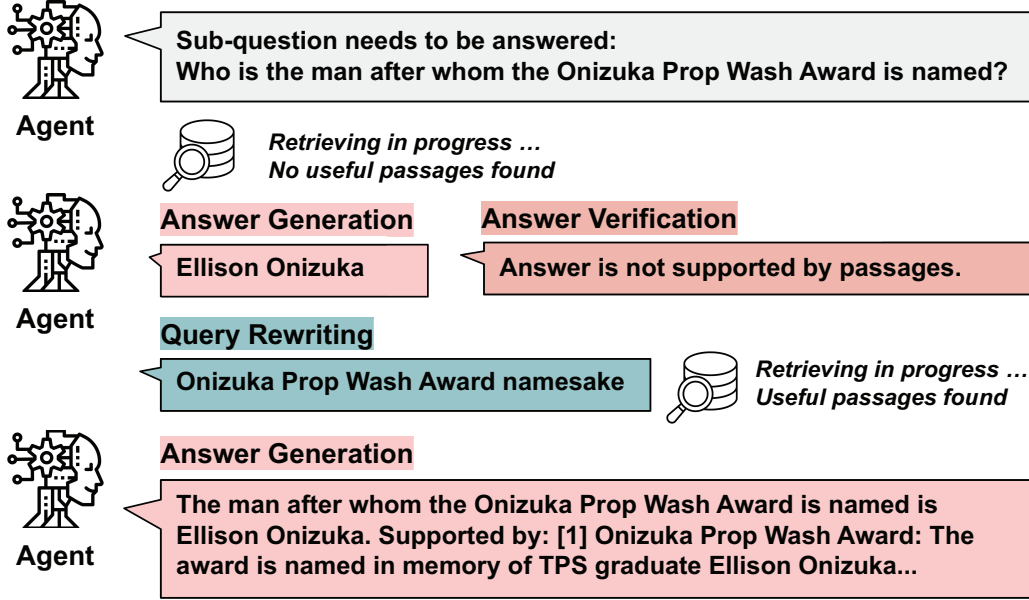


Figure 4: Iterative Refinement Example: Answer Verification and Query Rewriting

The **Simple QA Pipeline** is designed for straightforward questions that can typically be resolved through a single retrieval and generation cycle. This pipeline first retrieves relevant passages, then employs the Answer Generation module to synthesize an answer, and finally utilizes the Answer Verification module to validate its correctness and grounding. If the answer is successfully verified, it is returned to the user. If verification fails, or if the question is initially assessed as too complex for this direct approach, the system may escalate the query to the more comprehensive Multi-Hop QA Pipeline. Further implementation details of this streamlined pipeline are available in Appendix A.2.

For complex questions that necessitate multi-step reasoning and the integration of information from multiple sources, ComposeRAG employs the **Advanced Multi-Hop QA Pipeline**, the general flow of which is illustrated in Fig. 1. This pipeline begins with the **Initial Decomposition** of the original complex question into a sequence of simpler sub-questions by the Question Decomposition module. Each of these sub-questions then undergoes an **Iterative Sub-Question Processing** cycle. Within this cycle, the Question Construction module first formulates an actionable query from the current sub-question and any previously derived answers. The Retrieval Decision module then determines if external evidence is required. If so, passages are retrieved—potentially enhanced by the Query Rewriting module for ambiguous or context-dependent sub-questions—and subsequently prioritized by the Passage Reranking module. Following this, the Answer Generation module produces a candidate sub-answer based on the provided evidence. Crucially, this sub-answer is then scrutinized by the Answer Verification module. If verification fails (e.g., due to insufficient evidence or a poorly formulated sub-query leading to irrelevant retrieval), this can trigger corrective actions, as demonstrated in the iterative refinement example (Fig. 4).

Once all sub-questions have been successfully addressed and their answers verified through this iterative process, the **Final Synthesis** step occurs, where the Final Answering module aggregates the intermediate results into a single, comprehensive answer to the original multi-hop query. This structured, iterative, and verification-centric process is designed to enhance both the transparency and the robustness of the multi-hop reasoning.

4.3 Orchestration: Self-Reflection

In composable design, orchestration plays a critical role, and orchestration should be **progress-informed**, to adapt based on module outputs, or partial evaluation signals, for updating orchestration policies towards improving downstream performance.

For example, if the initial breakdown of a complex question is flawed, these errors may propagate through subsequent steps, potentially leading to an incorrect or ungrounded final answer. Desirably, orchestration should be informed and proactively mitigate this, ComposeRAG incorporates a **Self-Reflection Mechanism**. This mechanism allows the entire pipeline to revisit and refine its reasoning process if the final synthesized answer fails a concluding verification check (the overall process is summarized in Algorithm. 1).

The self-reflection process operates through several stages. First, the final answer produced by the Multi-Hop QA Pipeline undergoes a rigorous check by the Answer Verification module. If this final verification fails, the system initiates an **Error Analysis and Diagnosis** phase. During this phase, it analyzes the complete reasoning trace, which includes the initial decomposition, all intermediate sub-questions, the retrieved passages, and the generated sub-answers, to identify the most probable source of the error. Often, such errors can be traced back to a suboptimal initial decomposition. Based on this diagnosis, specialized prompting techniques are used to generate **Guided Re-decomposition** instructions. These instructions provide corrective feedback to the Question Decomposition module, guiding it on how to improve the initial breakdown of the original question. The Question Decomposition module then re-processes the original question, now informed by this targeted feedback, to produce a revised set of sub-questions. Finally, the Multi-Hop QA Pipeline is **Re-executed** using this new, hopefully improved, set of sub-questions.

This capacity for iterative refinement at the pipeline level allows ComposeRAG to learn from its mistakes within the context of a single problem-solving instance. By diagnosing and correcting flaws in its own reasoning process, particularly in the critical decomposition phase, the self-reflection mechanism significantly improves the reliability and accuracy of the final answers, demonstrating the adaptive power inherent in ComposeRAG’s modular design.

Algorithm 1 Self-Reflection for Decomposition Improvement

```

Specification: Final answer  $a_{\text{final}}$ , question  $q$ , supporting passages  $\mathcal{P}$ , question solving record  $H$ 
 $(y, r) \leftarrow \text{Verify}_{\theta}(q, a_{\text{final}}, \mathcal{P})$  ▷ Step 1: Final answer verification
if  $y = \text{True}$  then
  return  $a_{\text{final}}$  ▷ Answer is correct
else
   $\text{analysis} \leftarrow \text{ImproveAnalysis}_{\theta}(q, H)$  ▷ Step 2: Analyze the solving record
   $\mathbf{S}_{\text{new}} \leftarrow \text{ImproveDecomposition}_{\theta}(q, \text{analysis})$  ▷ Step 3: Generate new decomposition
   $a_{\text{new}} \leftarrow \text{MultiHopQAPipeline}(q, \mathbf{S}_{\text{new}})$  ▷ Step 4: Rerun the pipeline
  return  $a_{\text{new}}$ 
end if

```

5 Experiments

5.1 Experimental Setup

Datasets and Retrieval Corpus: We evaluate ComposeRAG on four widely-used multi-hop question answering datasets: HotpotQA [32], 2WikiMultiHopQA [10], MuSiQue [25], and Bamboogle [20]. These datasets are designed to test complex reasoning capabilities by requiring models to integrate information from multiple sources. For HotpotQA, 2WikiMultiHopQA, and MuSiQue, we randomly sampled 500 questions from their respective dev sets for evaluation. For Bamboogle, we used the entire test set comprising 125 questions. For open-domain retrieval, we utilize the English Wikipedia corpus as the background knowledge source. We employ two versions of this corpus:

- A truncated version of the 2018-12-20 Wikipedia dump, as released by FlashRAG [14], where articles are segmented into short chunks.
- A version pre-processed by us using the 2019-08-01 Wikipedia dump from the KILT knowledge source [19]. In our pre-processing, we retain the original abstract of each article and segment the main body into chunks of 4-5 sentences to ensure consistency and relevance for dense retrieval.

Baselines:

We aim to evaluate multi-hop QA systems within a modular reasoning framework that emphasizes transparency, component-level control, and iterative verification. To this end, we focus on systems that align with these design principles and are compatible with our evaluation protocol. We select **RQ-RAG** and **Search-o1** as our primary baselines for reproduction and comparison. These two methods represent complementary paradigms—RQ-RAG employs supervised fine-tuning for query refinement, while Search-o1 leverages zero-shot agentic prompting with large language models.

- **RQ-RAG [5]:** We follow the setup in FlashRAG [14], using e5-base-v2 as the embedding model, to reproduce its results.
- **Search-o1 [18]:** It originally relies on Bing Search combined with Jina AI for real-time web retrieval. To ensure a fair comparison with our system focused on a local Wikipedia corpus, we adapt its retrieval mechanism to use the same local corpus as ComposeRAG.

Although we do not re-implement reinforcement learning-based methods, we include **ReARTeR** [23] and **R1-Searcher** [21] by reporting their published results, as they share similar evaluation setups and address related reasoning objectives. **Search-R1** [13], on the other hand, focuses on learning reasoning and retrieval policies via reinforcement learning with relatively small-scale models, and adopts a tightly coupled end-to-end architecture. As its design goals and evaluation protocol differ substantially from ours, we do not include it in the main set of empirical comparisons, but discuss it in related work for completeness.

Implementation Settings: For LLM-based generation within ComposeRAG, we primarily use three series of models:

- GPT-based models (e.g., GPT-4o, GPT-4o-mini) accessed via the Azure OpenAI Service, providing reliable and scalable inference. (Note: Citations for GPT-4o and mini might be more recent than [1] for GPT-4, adjust if necessary).
- Models based on LLaMA 3.1 [7], accessed using the Cortex Completion Service for efficient inference with open-weight models.
- Locally deployed models such as QwQ-32B [35] and Qwen2.5 [31], using the vLLM framework on 8xV100 GPUs.

For dense retrieval, ComposeRAG supports two configurations:

- The e5-base-v2 model [27] to encode queries and passages into dense vector representations, with indexing handled by FAISS [6] for efficient similarity-based retrieval.
- The Cortex Search API, which utilizes snowflake-arctic-embed-m-v1.5 as its default embedding model.

Evaluation Metrics: We employ two primary metrics to evaluate performance:

- **LLM Evaluation:** We use GPT-4o as an automatic evaluator to judge whether a predicted answer semantically aligns with the ground truth. The evaluation is guided by a structured prompt that considers relevance, semantic consistency, and completeness. The model outputs true if the answer matches any ground truth in meaning and contains all key information, and false otherwise. This setup aligns with recent practices in using LLMs as evaluators [8].
- **Cover Exact Match (Cover EM):** This metric is a relaxed variant of the standard Exact Match. Let $\hat{a}$ be the predicted answer and $\mathcal{A} = \{a_1, a_2, \dots, a_k\}$ be the set of ground truth answers. For each answer string, we apply a normalization function, $\text{Normalize}()$, which lowercases the text, removes punctuation, trims whitespace, and tokenizes the result. Let $T_{\hat{a}} = \text{Normalize}(\hat{a})$ be the tokenized prediction, and $T_{a_j} = \text{Normalize}(a_j)$ be the tokenized form of each ground truth $a_j \in \mathcal{A}$. The function $\text{Subseq}(s, t)$ returns 1 if s is a contiguous subsequence of t , and 0 otherwise. Cover EM is then computed as:

$$\text{CoverEM}(\hat{a}, \mathcal{A}) = \begin{cases} 1, & \text{if } \exists a_j \in \mathcal{A} \text{ such that } \text{Subseq}(T_{a_j}, T_{\hat{a}}) = 1 \\ 0, & \text{otherwise} \end{cases}$$

This metric accounts for predictions that contain longer or more natural language expressions, as long as the essential answer span from a ground truth appears intact within the prediction.

5.2 Empirical Results and Analysis

This section presents the empirical evaluation of ComposeRAG, focusing on its performance against established baselines and the scalability of its modular design. The primary results across four multi-hop QA datasets are summarized in Table. 1. A detailed discussion regarding model selection, the compatibility of models with our pipeline’s design, and the strategy for ensuring fair comparisons—including the alignment of retrieval corpora and LLM pairings—is provided in Appendix B.

Table 1: Results table for four multi-hop datasets with different retrieval and generation setting

HotpotQA						
Method	LLM	Retriever	Corpus	LLM Eval	Cover-EM	Avg
RQ-RAG	fine-tuned llama2-7b	e5-base-v2	wiki2018	0.326	0.258	0.292
Ours	Llama3.1-8b	Cortex Search	wiki2018	0.494	0.388	0.441
Search-o1	QwQ-32B	Cortex Search	wiki2019	0.664	0.492	0.578
Ours	Qwen2.5-72B-Instruct	Cortex Search	wiki2019	0.714	0.554	0.634
Ours	GPT-4o-mini	Cortex Search	wiki2019	0.702	0.558	0.630
Ours	GPT-4o	Cortex Search	wiki2019	0.732	0.580	0.656
2WikiMultiHopQA						
Method	LLM	Retriever	Corpus	LLM Eval	Cover-EM	Avg
RQ-RAG	fine-tuned llama2-7b	e5-base-v2	wiki2018	0.240	0.268	0.254
Ours	Llama3.1-8b	Cortex Search	wiki2018	0.326	0.334	0.330
Search-o1	QwQ-32B	Cortex Search	wiki2019	0.808	0.770	0.789
Ours	Qwen2.5-72B-Instruct	Cortex Search	wiki2019	0.720	0.726	0.723
Ours	GPT-4o-mini	Cortex Search	wiki2019	0.740	0.728	0.734
Ours	GPT-4o	Cortex Search	wiki2019	0.820	0.808	0.814
MuSiQue						
Method	LLM	Retriever	Corpus	LLM Eval	Cover-EM	Avg
RQ-RAG	fine-tuned llama2-7b	e5-base-v2	wiki2018	0.096	0.086	0.091
Ours	Llama3.1-8b	Cortex Search	wiki2018	0.128	0.102	0.115
Search-o1	QwQ-32B	Cortex Search	wiki2019	0.374	0.278	0.326
Ours	Qwen2.5-72B-Instruct	Cortex Search	wiki2019	0.358	0.318	0.338
Ours	GPT-4o-mini	Cortex Search	wiki2019	0.376	0.328	0.352
Ours	GPT-4o	Cortex Search	wiki2019	0.388	0.342	0.365
Bamboogle						
Method	LLM	Retriever	Corpus	LLM Eval	Cover-EM	Avg
RQ-RAG	fine-tuned llama2-7b	e5-base-v2	wiki2018	0.264	0.224	0.244
Ours	Llama3.1-8b	Cortex Search	wiki2018	0.432	0.384	0.408
Search-o1	QwQ-32B	Cortex Search	wiki2019	0.696	0.648	0.672
Ours	Qwen2.5-72B-Instruct	Cortex Search	wiki2019	0.728	0.616	0.672
Ours	GPT-4o-mini	Cortex Search	wiki2019	0.712	0.640	0.676
Ours	GPT-4o	Cortex Search	wiki2019	0.728	0.624	0.676

5.2.1 Comparison with RQ-RAG: General-Purpose Prompting vs. Task-Specific Fine-Tuning

We first compare ComposeRAG, when utilizing an instruction-tuned Llama3.1-8B model in a prompted configuration, against RQ-RAG, which employs a Llama2-7B model fine-tuned for the task. As evidenced in Table. 1, ComposeRAG demonstrates superior performance across all evaluated datasets without resorting to task-specific fine-tuning. Notably, on the HotpotQA and Bamboogle datasets, ComposeRAG achieves substantial improvements in both LLM-based evaluation scores and Cover EM. For instance, on HotpotQA, ComposeRAG (Llama3.1-8B) achieves an average score of 0.441, compared to 0.292 for RQ-RAG. This outcome highlights the efficacy of

leveraging general-purpose, pre-trained instruction-tuned models within a well-structured modular prompting framework. Furthermore, these results underscore the inherent flexibility and scalability of the ComposeRAG design, which circumvents the need for resource-intensive custom training pipelines for individual components or tasks, aligning with our goal of creating an adaptable and broadly applicable system.

5.2.2 Comparison with Search-o1: Grounded Modular Reasoning vs. Ungrounded Specialized Reasoning

Next, we benchmark ComposeRAG, configured with the Qwen2.5-72B-Instruct model, against Search-o1, which utilizes the reasoning-specialized QwQ-32B model. To ensure an equitable comparison, both systems operate with the identical retriever (Cortex Search) and retrieval corpus (wiki2019). The results presented in Table. 1 indicate that ComposeRAG achieves performance levels that are either superior to or comparable with Search-o1 across the majority of datasets. Specifically, on HotpotQA, ComposeRAG (Qwen2.5-72B) shows an average score of 0.634 against Search-o1’s 0.578. Similarly, on Bamboogle, ComposeRAG achieves an average of 0.672, matching Search-o1’s performance. On MuSiQue, ComposeRAG demonstrates higher answer coverage (Cover-EM of 0.318 vs. 0.278), although its LLM evaluation score is marginally lower.

The primary exception to ComposeRAG’s strong performance is on 2WikiMultiHopQA, where Search-o1 outperforms it (average of 0.789 vs. 0.723 with Qwen2.5-72B). This dataset often involves implicit reasoning and sparse evidence, where Search-o1 tends to produce plausible but not always well-grounded answers. In contrast, ComposeRAG emphasizes factual grounding through verification modules, which enhances correctness but can lower answer rates when sufficient supporting context is not retrieved. This trade-off is particularly relevant in applications requiring verifiable outputs.

To better understand this behavior, we analyze cases where Search-o1 succeeds but ComposeRAG fails. We find that approximately 18% of Search-o1-only correct answers lack direct support from the retrieved context. These answers are plausible but not explicitly grounded in evidence, raising concerns about their reliability. When averaged over the full dataset, these ungrounded yet correct responses account for 2.8% of all evaluated examples. This analysis highlights a key tradeoff in our system: by prioritizing evidence-based verification, we may sacrifice coverage in favor of factual precision. Appendix C provides detailed examples and categorization of these grounding issues.

5.2.3 Scalability Across Model Sizes: GPT-4o and GPT-4o-mini

To evaluate how ComposeRAG scales with the underlying model capacity, we test it using both GPT-4o and its lighter version, GPT-4o-mini. As shown in Table. 1, increased model size consistently yields higher performance—GPT-4o achieves the top scores across all datasets, with average scores such as 0.656 on HotpotQA and 0.814 on 2WikiMultiHopQA, compared to 0.630 and 0.734 for GPT-4o-mini, respectively. Despite its reduced size, GPT-4o-mini remains highly competitive, frequently outperforming strong baselines like Search-o1. For instance, on 2WikiMultiHopQA, it achieves a Cover-EM score of 0.728, closely matching or exceeding results from larger, task-specialized models. These findings demonstrate that our modular pipeline generalizes robustly across model scales, delivering strong multi-hop QA performance even under constrained computational budgets.

Table 2: Comparison with prior work on Multi-Hop QA benchmarks. Each pair of values shows LLM Eval (left) and Cover-EM (right). All values are reported from the original papers. Evaluation settings may differ in aspects such as sample selection, prompt format, retriever, and passage truncation.

Method	HotpotQA	2WikiMultiHopQA	MuSiQue	Bamboogle	Avg
ReARTeR [23]	(0.506 / 0.468)	(0.534 / 0.554)	(0.302 / 0.296)	(0.544 / 0.496)	0.463
R1-Searcher [21]	(0.750 / 0.654)	(0.650 / 0.636)	(0.314 / 0.282)	(0.544 / 0.528)	0.545
ComposeRAG	(0.702 / 0.558)	(0.740 / 0.728)	(0.376 / 0.328)	(0.712 / 0.640)	0.598

5.2.4 Comparison with other Prior Works

To further contextualize the performance of ComposeRAG under the GPT-4o-mini setting, we compare against retrieval-augmented baselines in Table. 2. **ReARTeR** [23] uses GPT-4o-mini as its generator, while **R1-Searcher** [21] employs an RL-enhanced Qwen-2.5-7B. All values reflect the best results reported in their original papers. As shown, ComposeRAG achieves the highest LLM Eval and Cover-EM scores on three of the four benchmarks—2WikiMultiHopQA, MuSiQue, and Bamboogle—trailing only R1-Searcher on HotpotQA. These findings highlight the strength of our modular design: it enables general-purpose models to perform competitively or even outperform fine-tuned, task-specific alternatives, despite using lighter model configurations.

6 Discussion and Ablation

To rigorously evaluate the efficacy of our modular design and the contributions of its constituent components, we conduct a series of ablation studies. These studies are performed on a randomly sampled subset of 200 examples from both the HotpotQA and MuSiQue datasets. For these ablation experiments, GPT-4o-mini is primarily employed as the base language model to ensure consistent and controlled comparisons.

6.1 Effect of Self-Reflection on Multi-Hop QA Performance

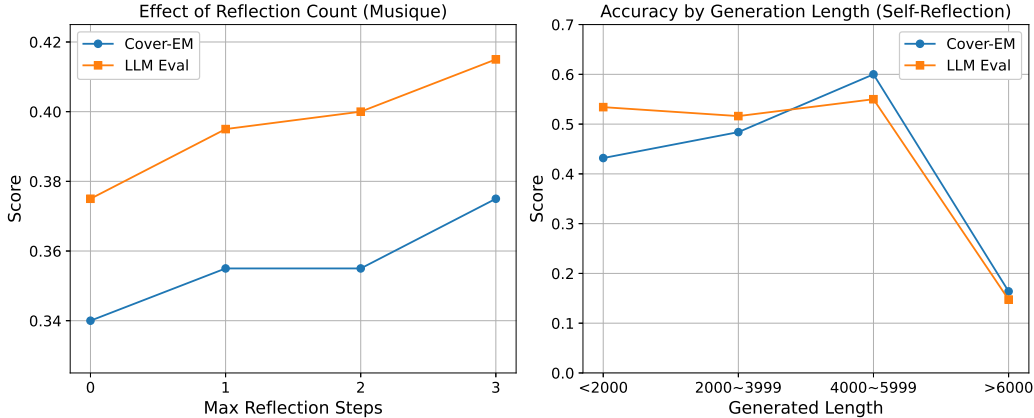


Figure 5: Accuracy of the self-reflection mechanism on the Musique dataset. **Left:** Accuracy improves as the number of reflection steps increases. **Right:** Performance by total generation length.

To enhance the robustness of multi-hop reasoning, ComposeRAG incorporates a self-reflection mechanism. This feature permits the system to iteratively re-attempt the reasoning process up to a maximum of three times if the initial answer fails verification. The reflection process terminates early if a high-confidence answer is successfully verified. A configuration with zero reflection steps serves as the baseline, representing either a single pass through the multi-hop pipeline or an early exit via the Simple QA pathway.

As illustrated in Figure 5 (Left), the iterative self-reflection mechanism yields significant performance improvements on the Musique dataset. With each incremental reflection step, both Cover-EM and LLM Eval scores exhibit a steady increase. Specifically, Cover-EM rises from 0.340 (at 0 reflections) to 0.375 (at 3 reflections), while the LLM Eval score improves from 0.375 to 0.415 over the same range. This progression confirms that revisiting and refining earlier stages of the reasoning process enables the model to effectively correct initial errors and enhance the quality of its answers.

Figure 5 (Right) analyzes performance as a function of the total generation length, aggregated across all reflection steps. Optimal accuracy is observed for generations within the 4000–5999 token range, suggesting that moderately lengthy reflective reasoning is most efficacious. Conversely, a sharp decline in performance is noted for outputs exceeding 6000 tokens. This decline is likely attributable to unresolved complexities in the underlying questions or instances of reasoning drift during prolonged iterative processing, rather than an inherent flaw in the reflection mechanism itself.

In summary, these results demonstrate that bounded self-reflection is a valuable technique for improving multi-hop QA performance by facilitating iterative error correction. While extremely long generation traces correlate with lower accuracy, this primarily reflects the intrinsic difficulty of those specific queries.

6.2 Ablation: Assessing the Contribution of Individual Modules

Table 3: Module effectiveness for Multi-Hop QA

Modules					Hotpot-200		Musique-200	
QD	QC	QR	PR	AV	Cover-EM	LLM Eval	Cover-EM	LLM Eval
✗	✗	✗	✗	✗	0.480	0.625	0.170	0.195
✓	✓	✗	✗	✗	0.505	0.650	0.320	0.360
✓	✓	✗	✓	✗	0.510	0.665	0.335	0.365
✓	✓	✓	✗	✓	0.510	0.660	0.330	0.365
✓	✓	✓	✓	✓	0.505	0.680	0.350	0.375

To empirically validate the contribution of individual components within our modular architecture, as outlined in our key contributions (Section 1), we conducted an ablation study assessing the incremental impact of five core modules: Question Decomposition (QD), Question Construction (QC), Passage Reranking (PR), Query Rewriting (QR), and Answer Verification (AV). The results of this stepwise module integration, presented in Table 3, are measured using both Cover-EM and LLM-based evaluation. It is important to note certain inherent couplings in module functionality: QD and QC are necessarily paired, as decomposition generates sub-questions that require subsequent construction into executable queries. Similarly, QR is typically activated following verification failures, making it closely integrated with AV. Our evaluation strategy respects these dependencies.

The analysis commences with a baseline configuration where no specialized modules are active, resulting in limited performance, particularly on the more complex MuSiQue dataset (0.170 Cover-EM, 0.195 LLM Eval). The introduction of the foundational QD and QC modules yields the most substantial performance improvements. For instance, on MuSiQue, this addition nearly doubles the Cover-EM to 0.320 and significantly boosts the LLM Eval score to 0.360, underscoring the critical role of structured decomposition and sub-question formulation in multi-hop reasoning. The subsequent integration of the PR module, which enhances context quality through improved evidence ranking, provides further incremental gains (e.g., on HotpotQA, LLM Eval increases from 0.650 to 0.665 when PR is added to the QD and QC configuration). Finally, the inclusion of the QR and AV modules, which refine intermediate reasoning steps and filter unsupported answers, contributes to further enhancements, particularly in the LLM evaluation metric (e.g., on MuSiQue, LLM Eval rises from 0.365 with QD, QC, and PR to 0.375 when all modules are active). These findings collectively confirm that each module provides a distinct and valuable contribution to the overall pipeline performance. The initial decomposition and construction modules establish a strong foundation, while subsequent modules for reranking, rewriting, and verification progressively refine the reasoning process, enhancing answer precision and robustness.

6.3 Upgrade: Independent Contribution of Upgraded Modules

Table 4: Impact of individually upgrading modules

QD	QC	QR	PR	AG	AV	Cover-EM	LLM Eval
○	○	○	○	○	○	0.350	0.375
✓	○	○	○	○	○	0.350	0.390
✓	✓	○	○	○	○	0.355	0.395
✓	✓	✓	○	○	○	0.355	0.405
✓	✓	✓	✓	○	○	0.380	0.420
✓	✓	✓	✓	✓	○	0.380	0.430
✓	✓	✓	✓	✓	✓	0.385	0.450

A key tenet of ComposeRAG, as highlighted in the Introduction, is the independent upgradability of its modules, contributing to the framework’s flexibility and scalability. To empirically demonstrate

this, we investigate the standalone benefit of enhancing individual modules by comparing two operational versions: a baseline configuration where each module is implemented using `gpt-4o-mini` (denoted as ○), and an enhanced version where a specific module is upgraded to use the more powerful `gpt-4o` model (denoted as ✓). The Answer Generation (AG) module, a constant presence in the pipeline, is also subject to this upgradability. These experiments were conducted on the Musique-200 subset.

The study begins with a baseline where all modules utilize `gpt-4o-mini`, establishing an LLM Eval score of 0.375. We then progressively upgrade one module at a time to its `gpt-4o` counterpart. As detailed in Table. 4, upgrading the Question Decomposition (QD) module alone results in an immediate increase in the LLM Eval score from 0.375 to 0.390. Subsequent individual upgrades to other modules, such as Question Construction (QC), Query Rewriting (QR), Passage Reranking (PR), Answer Generation (AG), and Answer Verification (AV), each contribute to further, steady improvements in performance. When all modules are upgraded to utilize `gpt-4o`, the LLM Eval score reaches 0.450. These results clearly confirm that enhancing individual modules in isolation yields measurable and additive performance gains. This validates the independent upgradability feature of ComposeRAG, underscoring its capacity to adapt to varying computational resources and to benefit from advancements in individual model capabilities without requiring a complete overhaul of the pipeline. This inherent flexibility and scalability are central to the practical utility of our framework.

6.4 Efficiency-Oriented Design: Retrieval Decision and Simple QA Handling

Table 5: Impact of Retrieval Decision on Hotpot-200

Decision	Cover-EM	LLM Eval
no	0.540	0.725
gpt-4o-mini	0.535	0.745
gpt-4o	0.555	0.745

Table 6: Impact of adding Simple QA on Hotpot-200 (`gpt-4o-mini`)

Simple QA	Cover-EM	LLM Eval	Avg. Tokens
no	0.505	0.680	1219
yes	0.530	0.695	832

To bolster computational efficiency without unduly compromising accuracy, ComposeRAG incorporates two dynamic components: the Retrieval Decision module and a dedicated Simple QA pipeline. The Retrieval Decision module intelligently determines whether external knowledge retrieval is necessary for a given (sub-)question. This allows the system to bypass the retrieval step if the input context already contains sufficient information for grounded reasoning, thereby conserving resources. It is important to note that this decision does not imply a fallback to ungrounded parametric knowledge; rather, it prioritizes leveraging already available, contextualized information. Complementing this, the Simple QA module is designed to identify and directly process questions that do not require complex multi-hop reasoning. By routing such "easy" questions through a lightweight path, it avoids the computational overhead associated with the full multi-hop pipeline.

The efficacy of these efficiency-oriented components was evaluated on the Hotpot-200 subset, using both `gpt-4o-mini` and `gpt-4o` as the backbone LLM. As indicated in Table. 5, activating the Retrieval Decision module (comparing "no" decision to using `gpt-4o-mini` or `gpt-4o` for the decision) generally maintains or slightly improves performance. For instance, with `gpt-4o-mini`, the LLM Eval score increases from 0.725 (no decision module) to 0.745 when the decision module is active. The optimal configuration, employing `gpt-4o` for the decision, achieves the highest Cover-EM of 0.555 while matching the top LLM Eval score of 0.745.

Furthermore, Table. 6 demonstrates the benefits of integrating the Simple QA pipeline. When this pathway is enabled for the Hotpot-200 subset (using `gpt-4o-mini`), there is a marked reduction in average token consumption per query, decreasing by approximately 32% (from 1219 to 832 tokens). Concurrently, both Cover-EM (from 0.505 to 0.530) and LLM Eval (from 0.680 to

0.695) scores exhibit improvement. These results collectively underscore the advantages of ComposeRAG’s flexible, modular design, where adaptive routing mechanisms like early exits for simple questions and conditional retrieval can be seamlessly integrated to optimize for both efficiency and effectiveness. seamlessly integrated to improve efficiency without compromising effectiveness.

6.5 Impact of Generation Length on Accuracy

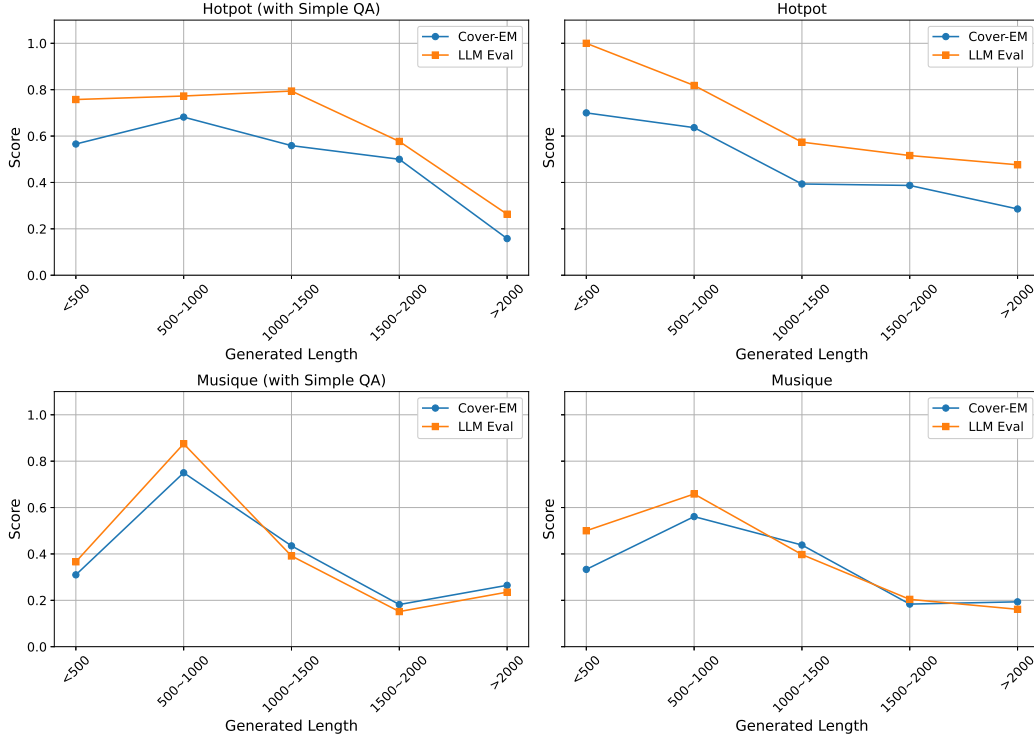


Figure 6: Accuracy across different generation length intervals. Each subplot shows the relationship between output length and performance for Hotpot and Musique.

To investigate the relationship between the length of generated outputs and system performance, we conducted an analysis where model outputs were categorized based on their total generated token count. These generations were divided into five distinct length intervals. Within each interval, we compared Cover-EM and LLM evaluation scores for two configurations of our system: the standard Multi-Hop QA pipeline and an augmented version incorporating the Simple QA pipeline.

The results, depicted in Figure 6, reveal a general trend where longer generations are associated with lower accuracy. Specifically for the HotpotQA dataset, the Cover-EM score decreases markedly from 0.700 for outputs containing fewer than 500 tokens to a mere 0.286 for those exceeding 2000 tokens. This suggests that extended outputs often correspond to more complex or unresolved queries, potentially reflecting a more diffuse or less successful reasoning process. A similar pattern is observable for the MuSiQue dataset, although the relationship appears more nuanced, likely due to the inherently higher difficulty of this benchmark. In this context, very short generations might indicate premature termination of the reasoning process, whereas outputs of moderate length tend to correlate with optimal performance.

The introduction of the Simple QA pipeline demonstrates a positive impact on performance for mid-length generations, primarily by enabling efficient early exits for questions that are solvable without extensive multi-hop reasoning. However, a slight decrease in accuracy is noted for shorter generations on the MuSiQue dataset when the Simple QA pipeline is active. This may suggest instances where the system occasionally misclassifies complex queries as simple, leading to underperformance due to premature routing.

Overall, this analysis highlights the intricate connection between generation length, the inherent difficulty of the input question, and the behavior of the pipeline. Monitoring the length of generated outputs can therefore serve as a valuable heuristic for refining routing decisions and verification strategies within modular question answering systems like ComposeRAG.

7 Conclusion

In this work, we presented a modular abstraction for Multi-Hop RAG that addresses the limitations of existing monolithic QA pipelines. By decomposing the reasoning process into a sequence of parameterized, composable modules, our framework promotes interpretability, supports component-wise analysis, and enables flexible reconfiguration for diverse tasks. Across four multi-hop QA benchmarks, our modular pipeline consistently outperforms strong baselines like RQ-RAG and Search-o1, while maintaining scalability across large language models of varying capacities. Notably, our system enhances factual grounding through verification and abstention mechanisms, and recovers from errors via iterative self-reflection. These results demonstrate that a modular design can match or exceed the performance of specialized systems while offering greater transparency, adaptability, and extensibility for future RAG development.

8 Limitations

Despite its strengths, our approach has several limitations. First, the performance of the overall system is tightly coupled to the reliability of individual modules, particularly the Question Decomposition module. When questions exhibit subtle logic or ambiguous structure, decomposition may still fail, even with self-reflective refinement. Second, our method relies on large instruction-tuned LLMs for module execution, which introduces latency and resource overhead. This may limit the framework’s applicability in low-resource or real-time settings. Future work may explore lighter-weight alternatives or dynamic adaptation strategies to reduce inference cost while preserving modular reasoning quality.

References

- [1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altschmidt, Sam Altman, Shyamal Anadkat, et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.
- [2] Jayetri Bardhan, Bushi Xiao, and Daisy Zhe Wang. Ttqa-rs-a break-down prompting approach for multi-hop table-text question answering with reasoning and summarization. *arXiv preprint arXiv:2406.14732*, 2024.
- [3] Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George Bm Van Den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, et al. Improving language models by retrieving from trillions of tokens. In *International conference on machine learning*, pages 2206–2240. PMLR, 2022.
- [4] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.
- [5] Chi-Min Chan, Chunpu Xu, Ruibin Yuan, Hongyin Luo, Wei Xue, Yike Guo, and Jie Fu. Rq-rag: Learning to refine queries for retrieval augmented generation. *arXiv preprint arXiv:2404.00610*, 2024.
- [6] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. The faiss library. *arXiv preprint arXiv:2401.08281*, 2024.
- [7] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

- [8] Jiawei Gu, Xuhui Jiang, Zhichao Shi, Hexiang Tan, Xuehao Zhai, Chengjin Xu, Wei Li, Yinghan Shen, Shengjie Ma, Honghao Liu, et al. A survey on llm-as-a-judge. *arXiv preprint arXiv:2411.15594*, 2024.
- [9] Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Mingwei Chang. Retrieval augmented language model pre-training. In *International conference on machine learning*, pages 3929–3938. PMLR, 2020.
- [10] Xanh Ho, Anh-Khoa Duong Nguyen, Saku Sugawara, and Akiko Aizawa. Constructing a multi-hop qa dataset for comprehensive evaluation of reasoning steps. *arXiv preprint arXiv:2011.01060*, 2020.
- [11] Gautier Izacard, Patrick Lewis, Maria Lomeli, Lucas Hosseini, Fabio Petroni, Timo Schick, Jane Dwivedi-Yu, Armand Joulin, Sebastian Riedel, and Edouard Grave. Atlas: Few-shot learning with retrieval augmented language models. *Journal of Machine Learning Research*, 24(251):1–43, 2023.
- [12] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. Survey of hallucination in natural language generation. *ACM computing surveys*, 55(12):1–38, 2023.
- [13] Bowen Jin, Hansi Zeng, Zhenrui Yue, Jinsung Yoon, Sercan Arik, Dong Wang, Hamed Zamani, and Jiawei Han. Search-r1: Training llms to reason and leverage search engines with reinforcement learning. *arXiv preprint arXiv:2503.09516*, 2025.
- [14] Jiajie Jin, Yutao Zhu, Guanting Dong, Yuyao Zhang, Xinyu Yang, Chenghao Zhang, Tong Zhao, Zhao Yang, Zhicheng Dou, and Ji-Rong Wen. Flashrag: A modular toolkit for efficient retrieval-augmented generation research. *arXiv preprint arXiv:2405.13576*, 2024.
- [15] Jaehyung Kim, Jaehyun Nam, Sangwoo Mo, Jongjin Park, Sang-Woo Lee, Minjoon Seo, Jung-Woo Ha, and Jinwoo Shin. Sure: Summarizing retrievals using answer candidates for open-domain qa of llms. *arXiv preprint arXiv:2404.13081*, 2024.
- [16] Kiseung Kim and Jay-Yoon Lee. Re-rag: Improving open-domain qa performance and interpretability with relevance estimator in retrieval-augmented generation. *arXiv preprint arXiv:2406.05794*, 2024.
- [17] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems*, 33:9459–9474, 2020.
- [18] Xiaoxi Li, Guanting Dong, Jiajie Jin, Yuyao Zhang, Yujia Zhou, Yutao Zhu, Peitian Zhang, and Zhicheng Dou. Search-o1: Agentic search-enhanced large reasoning models. *arXiv preprint arXiv:2501.05366*, 2025.
- [19] Fabio Petroni, Aleksandra Piktus, Angela Fan, Patrick Lewis, Majid Yazdani, Nicola De Cao, James Thorne, Yacine Jernite, Vladimir Karpukhin, Jean Maillard, et al. Kilt: a benchmark for knowledge intensive language tasks. *arXiv preprint arXiv:2009.02252*, 2020.
- [20] Ofir Press, Muru Zhang, Sewon Min, Ludwig Schmidt, Noah A Smith, and Mike Lewis. Measuring and narrowing the compositionality gap in language models. *arXiv preprint arXiv:2210.03350*, 2022.
- [21] Huatong Song, Jinhao Jiang, Yingqian Min, Jie Chen, Zhipeng Chen, Wayne Xin Zhao, Lei Fang, and Ji-Rong Wen. R1-searcher: Incentivizing the search capability in llms via reinforcement learning. *arXiv preprint arXiv:2503.05592*, 2025.
- [22] Weiwei Sun, Lingyong Yan, Xinyu Ma, Shuaiqiang Wang, Pengjie Ren, Zhumin Chen, Dawei Yin, and Zhaochun Ren. Is chatgpt good at search? investigating large language models as re-ranking agents. *arXiv preprint arXiv:2304.09542*, 2023.

- [23] Zhongxiang Sun, Qipeng Wang, Weijie Yu, Xiaoxue Zang, Kai Zheng, Jun Xu, Xiao Zhang, Song Yang, and Han Li. Rearter: Retrieval-augmented reasoning with trustworthy process rewarding. *arXiv preprint arXiv:2501.07861*, 2025.
- [24] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.
- [25] Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. ♪ musique: Multihop questions via single-hop question composition. *Transactions of the Association for Computational Linguistics*, 10:539–554, 2022.
- [26] Prakhar Verma, Sukruta Prakash Midigeshi, Gaurav Sinha, Arno Solin, Nagarajan Natarajan, and Amit Sharma. Plan-rag: Planning-guided retrieval augmented generation. *arXiv preprint arXiv:2410.20753*, 2024.
- [27] Liang Wang, Nan Yang, Xiaolong Huang, Binxing Jiao, Linjun Yang, Daxin Jiang, Rangan Majumder, and Furu Wei. Text embeddings by weakly-supervised contrastive pre-training. *arXiv preprint arXiv:2212.03533*, 2022.
- [28] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022.
- [29] Jian Wu, Linyi Yang, Yuliang Ji, Wenhao Huang, Börje F Karlsson, and Manabu Okumura. Gendec: A robust generative question-decomposition method for multi-hop reasoning. *arXiv preprint arXiv:2402.11166*, 2024.
- [30] Wenhan Xiong, Xiang Lorraine Li, Srinu Iyer, Jingfei Du, Patrick Lewis, William Yang Wang, Yashar Mehdad, Wen-tau Yih, Sebastian Riedel, Douwe Kiela, et al. Answering complex open-domain questions with multi-hop dense retrieval. *arXiv preprint arXiv:2009.12756*, 2020.
- [31] An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, et al. Qwen2. 5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.
- [32] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W Cohen, Ruslan Salakhutdinov, and Christopher D Manning. Hotpotqa: A dataset for diverse, explainable multi-hop question answering. *arXiv preprint arXiv:1809.09600*, 2018.
- [33] Tian Yu, Shaolei Zhang, and Yang Feng. Auto-rag: Autonomous retrieval-augmented generation for large language models. *arXiv preprint arXiv:2411.19443*, 2024.
- [34] Jiajie Zhang, Shulin Cao, Tingjia Zhang, Xin Lv, Jiaxin Shi, Qi Tian, Juanzi Li, and Lei Hou. Reasoning over hierarchical question decomposition tree for explainable question answering. *arXiv preprint arXiv:2305.15056*, 2023.
- [35] Chujie Zheng, Zhenru Zhang, Beichen Zhang, Runji Lin, Keming Lu, Bowen Yu, Dayiheng Liu, Jingren Zhou, and Junyang Lin. Processbench: Identifying process errors in mathematical reasoning. *arXiv preprint arXiv:2412.06559*, 2024.
- [36] Ziyuan Zhuang, Zhiyang Zhang, Sitao Cheng, Fangkai Yang, Jia Liu, Shujian Huang, Qingwei Lin, Saravan Rajmohan, Dongmei Zhang, and Qi Zhang. Efficientrag: Efficient retriever for multi-hop question answering. *arXiv preprint arXiv:2408.04259*, 2024.

A Supplementary Modules and Pipeline Details

A.1 Detailed Modular Components

Question Decomposition Multi-hop questions often require synthesizing information spread across multiple documents or reasoning steps. To handle such complexity, we introduce this module that leverages a large language model to break down the original question into a sequence of interrelated sub-questions. Unlike prior approaches that structure reasoning through a tree [34] or graph-based decomposition [26], our method follows the format proposed in [25], prompting the model to generate sub-questions sequentially. This linear, interconnected formulation encourages the model to uncover and follow the underlying logical flow of the original question, ensuring each sub-question builds on previously retrieved or inferred information. Given an input question $q \in \mathcal{Q}$, the goal of the decomposition module is to generate two components: a set of sub-questions $\mathbf{S} = \{s_1, s_2, \dots, s_n\} \subset \mathcal{Q}$, where each s_i represents a single-hop question derived from q ; and a reasoning trace $r \in \mathcal{R}$, which provides the reasoning path needed to decompose q . Here, $\mathcal{Q}$ represents the space of all questions and $\mathcal{R}$ represents the space of all reasoning traces. Formally, we define the decomposition process as: $(\mathbf{S}, r) = \text{Decompose}_\theta(q)$, where Decompose_θ is a function parameterized by the LLM weights θ . This module enables downstream modules to independently retrieve evidence and give the answer.

Question Construction This is designed to rewrite an abstract or context-dependent sub-question into a fully specified, self-contained form. This is especially important in multi-hop question answering, where a sub-question may refer to previous answers (e.g., using placeholders like "#1" or pronouns like "he" or "it"). This module takes the following inputs:

- **Original Question** $q \in \mathcal{Q}$: The full multi-hop question that provides overall context.
- **Previous QA Pairs** $H = \{(s_1, r_1, a_1), \dots, (s_{i-1}, r_{i-1}, a_{i-1})\}$: The history of sub-questions and their corresponding answers with reasoning from earlier steps.
- **Current Sub-question** $s_i \in \mathcal{Q}$: The context-dependent sub-question that may contain references to earlier answers.

Using these inputs, the model constructs **Fully-Specified Sub-question** $\hat{s}_i \in \mathcal{Q}$: a rewritten version of s_i where all references are resolved based on the previous answers. So the construction process is defined as: $\hat{s}_i = \text{Construct}_\theta(q, H, s_i)$, where Construct_θ is a function parameterized by the LLM weights θ , mapping the original question q , previous QA history H , and current sub-question s_i to the resolved sub-question $\hat{s}_i$. This module ensures that all sub-questions are well-formed, explicit, and interpretable.

Retrieval Decision This module determines whether external retrieval is necessary for answering a given question. In some cases, the question already contains sufficient information for making a decision, such as comparison, reasoning, or arithmetic over known entities. By avoiding unnecessary retrievals, the system can reduce latency and improve efficiency while maintaining answer quality.

The retrieval decision process function is like: $(d, r) = \text{Decide}_\theta(q)$, where $q \in \mathcal{Q}$ is the input question, $d \in \{\text{True}, \text{False}\}$ is a binary decision indicating whether retrieval is needed, and $r \in \mathcal{R}$ is a reasoning trace justifying the decision. When $d = \text{False}$, the pipeline proceeds directly to reasoning or answering modules without invoking retrieval, otherwise will enable retrieval for the current question answering.

Query Rewriting In order to iteratively refine the search query to improve retrieval relevance and alignment with the user's intent, we have one module for query rewriting. At each step, it takes the original question and the last rewritten version of the query and generates a new, improved query that better captures the current information need. This approach draws inspiration from prior work [5], which demonstrates the effectiveness of intermediate query reformulation in multi-hop retrieval.

We define the query rewriting process as a function: $q' = \text{Rewrite}_\theta(q, q_{\text{prev}})$, where $q \in \mathcal{Q}$ is the original question, $q_{\text{prev}} \in \mathcal{Q}$ is the previous version of the rewritten query, and $q' \in \mathcal{Q}$ is the new query output. The function Rewrite_θ is implemented by a LLM with parameters θ , mapping the current and previous queries to an improved query formulation.

Passage Reranking Inspired by [22], this module aims to reorder a set of retrieved passages based on their relevance to a given search query. The reranking function considers both the semantic alignment between the query and each passage, and the informativeness of the content. Importantly, it also helps mitigate the lost-in-the-middle problem by promoting highly relevant passages to the top of the ranked list, since relevant evidence may be present but overlooked due to suboptimal ordering. We define the passage reranking process as: $(r, \pi) = \text{Rerank}_\theta(q, I, P)$, where $q \in \mathcal{Q}$ is the input search query, which may be either the original full question or a rewritten version of it. The set $I = \{i_1, i_2, \dots, i_k\}$ contains the numerical identifiers corresponding to the retrieved passages $P = \{p_1, p_2, \dots, p_k\}$. The function Rerank_θ , parameterized by LLM weights θ , returns a reasoning trace $r \in \mathcal{R}$ and a permutation $\pi \in \Pi$ over the identifiers I , representing the final ranked order. After post-processing for the output ranking order, we can get the full reranked passages for future use. The reranking plays a crucial role in ensuring that the most relevant evidence is prioritized for downstream reasoning and generation.

Answer Generation The Answer Generation module is responsible for producing the final answer to a given question by leveraging retrieved evidence and background information. It is designed to ensure that the answer is both accurate and grounded in the supporting context. This module takes three inputs:

- **Question q :** the current question, which may be either the original full question or a sub-question generated through decomposition.
- **Retrieved Passages $P = \{p_1, p_2, \dots, p_k\}$:** evidence passages retrieved from the knowledge corpus.
- **Background b :** contextual information to support reasoning. For an original full question, the background is set to "none". For a decomposed sub-question, the background includes prior sub-questions and their answers, helping to disambiguate and resolve incomplete context from earlier hops.

The output includes a reasoning trace $r \in \mathcal{R}$, which is an explanation that justifies the answer based on the input context and retrieved content; and an answer with citation $a \in \mathcal{A}$, which is a factual answer explicitly supported by the retrieved passages and includes citation markers referring to the evidence. We define the answer generation process as: $(a, r) = \text{AnswerGen}_\theta(q, P, b)$, where AnswerGen_θ is a generation function parameterized by the LLM weights θ , mapping the tuple (q, P, b) into the answer and its corresponding reasoning trace.

This setup enables the model to support both one-hop and multi-hop scenarios. In multi-hop settings, the background input b plays a critical role in maintaining consistency and grounding across steps, especially when earlier answers provide essential but non-retrievable information.

Answer Verification This module evaluates whether a generated answer is both (1) factually correct with respect to the original question, and (2) properly grounded in the retrieved support passages. The answer verification process is defined as a function: $(y, r) = \text{Verify}_\theta(q, a, P)$, where $q \in \mathcal{Q}$ is the input question, $a \in \mathcal{A}$ is the generated answer, and $P = \{p_1, p_2, \dots, p_k\}$ is the set of retrieved support passages. The output $y \in \{\text{True}, \text{False}\}$ is a binary label indicating whether the answer is valid, and $r \in \mathcal{R}$ is a reasoning trace justifying the verification outcome. The function Verify_θ is implemented by a LLM parameterized by weights θ . This module serves to filter out hallucinated or unsupported answers, and enforces strict grounding by requiring that every verified answer be explicitly justified using the retrieved content.

Final Answering This module is responsible for synthesizing the final answer to the original multi-hop question by integrating all intermediate results from the reasoning process. It takes as input the original question and an answer history, which includes the sequence of sub-questions, their corresponding reasoning traces, and their answers.

We define the final answering process as: $(a, r) = \text{Finalize}_\theta(q, H)$, where $q \in \mathcal{Q}$ is the original question, $H = \{(s_1, r_1, a_1), \dots, (s_n, r_n, a_n)\}$ is the answer history consisting of sub-questions s_i , reasoning traces r_i , and answers a_i , and Finalize_θ is a function parameterized by LLM weights θ . The output is the final answer $a \in \mathcal{A}$ and a comprehensive reasoning trace $r \in \mathcal{R}$ that explains the aggregation of intermediate steps.

This step completes the multi-hop pipeline by assembling fragmented insights into a single, well-justified final prediction.

A.2 Simple QA pipeline

This is designed to answer straightforward questions that can be resolved using a single retrieval step. First, the system retrieves a set of relevant passages $P = \{p_1, p_2, \dots, p_k\}$ from the document corpus using the retrieval function: $P = \text{Retrieve}(q)$. These passages, along with the question q , are passed to the **Answer Generation** module, which uses a language model to produce an answer $a \in \mathcal{A}$ and a reasoning trace $r \in \mathcal{R}$. Since the model will use the whole question to answer, the background input is set to none: $(a, r) = \text{AnswerGen}_\theta(q, P, \text{none})$. The candidate answer a is then validated by the **Answer Verification** module, which determines whether the answer is correct and grounded in the retrieved passages: $(y, r_v) = \text{Verify}_\theta(q, a, P)$. Here, $y \in \{\text{True}, \text{False}\}$ is a binary decision, and $r_v \in \mathcal{R}$ is a verification reasoning trace. Based on the verification result, the final output is determined as:

$$\text{FinalAnswer} = \begin{cases} a, & \text{if } y = \text{True} \\ \text{Answer from Multi-Hop QA pipeline}, & \text{if } y = \text{False} \end{cases} \quad (1)$$

If the answer is verified as correct, it is returned as the final output. Otherwise, the pipeline defers to a more advanced Multi-Hop QA pipeline for further processing. Simple QA pipeline ensures both efficiency and reliability by quickly resolving simple questions, but it encounters failures like retrieving only partial evidence from a single query or failing to identify implicit logical structure within the question. To address this, the question decomposition and construction design gives Multi-Hop QA Pipeline more advancement, which explicitly breaks the original question into a sequence of sub-questions and solve them step-by-step.

B Compatibility with Pipeline Design and Fair Comparison

For RQ-RAG, we use the fine-tuned llama2-7b model as in its original implementation. However, our system is designed for prompt-based modular reasoning without task-specific fine-tuning. We use Llama3.1-8b instead of llama2-7b for two reasons: (1) llama2-7b struggles to follow our structured prompts without fine-tuning, often yielding unparseable outputs; (2) Llama3.1-8b provides stronger instruction-following out of the box, making it more compatible with our pipeline. This reflects a broader contrast: RQ-RAG uses a fine-tuned small model, while our system demonstrates how general-purpose, instruction-tuned models can perform well via simple prompting, without any fine-tuning.

For Search-o1, we retain QwQ-32B, a reasoning-specialized model designed for multi-step inference and self-reflective answering. Replacing it with a general-purpose LLM would undermine its intended strengths. In contrast, our pipeline is designed to be model-agnostic and works best with instruction-following LLMs rather than reasoning-specialized engines. To ensure a meaningful comparison, we evaluate our system using a range of general-purpose LLMs chosen for both performance and compatibility with our prompt-based design. In particular, we use Qwen2.5-72B-Instruct, a large instruction-tuned model chosen to compare fairly with QwQ-32B, since reasoning-focused models are typically more efficient and capable at reasoning despite having fewer parameters. Moreover, we use GPT-4o to demonstrate how our pipeline scales with a more powerful model, highlighting its ability to fully leverage the capabilities of advanced LLMs. In addition, we include GPT-4o-mini, a significantly smaller variant, to show that our system maintains strong performance even with limited model capacity, reinforcing the pipeline’s efficiency, robustness, and adaptability across a wide range of compute budgets.

Importantly, we carefully control for retrieval to ensure fairness. RQ-RAG and our method both use the wiki2018 corpus when compared directly, as RQ-RAG is fine-tuned using retrieved knowledge from this version. In comparisons involving Search-o1, we use wiki2019 for both Search-o1 and our method, as it contains more up-to-date information, reducing bottlenecks in the retrieval stage and allowing both methods to demonstrate their full capabilities. Additionally, we fix the retriever to Cortex Search for both our system and Search-o1 to ensure consistent retrieval quality, while RQ-RAG uses its original e5-base-v2 retriever as part of its design.

C Grounding Analysis

C.1 Representative cases in wiki2019

Several representative cases illustrate the grounding issue. For example, in response to the question "*Who was born later, **Meinhart Maur** or **Slaviča Kundačina**?*", the retrieval contains the birth date of *Meinhart Maur* but provides no information about *Slaviča Kundačina*. However, the model identifies a similar name—*Slavica Čukteraš*, which assumes they are the same person, answering based on that speculative link. In another case, "*Who is the father of the director of the film **Les Truands**?*", the director is *Carlo Rim*, whose real name is *Jean Marius Richard*. Although no retrieved document directly names his father, the model infers it to be *Marius Richard*, reasoning from the director's birth name. Similarly, for "*What is the date of birth of the editor of **The Texas Ranger (Magazine)**?*", the retrieved context fails to identify the editor, but the model uses its internal knowledge to answer the candidate and then provides the corresponding birth date.

C.2 More detailed analysis in wiki2018

During the experiments on 2WikiMultiHopQA dataset, we found that many questions cannot be answered with the wiki2018 corpus due to lack of knowledge. We present the performance of our system and the Search-o1 baseline using wiki2018 in Table. 7. Among the examples that Search-o1 can generate correct answers but we give "I don't know." based on grounding consideration. We found the following issues which are summarized in Table. 8.

Assumption under insufficient information When the retrieved information is incomplete or ambiguous, the reasoning model may generate answers by making implicit assumptions. This behavior can lead to plausible-sounding but ungrounded responses. For example, consider the question: "*Which film has the director who died first, **Convoy Buddies** or **Tip Toes**?*" In the retrieved passages, no explicit information is available regarding the director of *Convoy Buddies*. However, the passages mention that *Paul L. Smith* starred in the film. The reasoning model incorrectly assumes that *Paul L. Smith* is the director and proceeds to retrieve and compare his death date *April 25, 2012* with that of the director of *Tip Toes*. While this produces a definitive answer, the conclusion is unsupported and results from speculative reasoning due to the lack of direct evidence.

Misidentification under semantic similarity The reasoning model may mistakenly align a query entity with a semantically similar but incorrect one when the exact match is not found in the retrieved content. For example, consider the question: "*Which album was released first, **L.A. To Miami** or **Today Is Our Valentine's Day**?*" In this case, there is no album or song with the exact title *Today Is Our Valentine's Day* in the corpus. However, the retrieved passages include information about the 2010 soundtrack album *Valentine's Day*, which was released on *February 9, 2010*. The model incorrectly interprets this as referring to the target entity and uses its release date to make a comparative judgment.

Factual hallucination without retrieval In some cases, the reasoning model produces factual claims that are not grounded in any retrieved evidence, instead relying on its internal parametric knowledge, which may be outdated or incorrect. For example, consider the question: "*Which film has the director who died earlier, **La Moglie Vergine** or **Melvin and Howard**?*" In this case, the model fails to initiate retrieval for information about the director of *Melvin and Howard*. Instead, it hallucinates an answer by incorrectly asserting that the director is *Michael Ritchie* and proceeds to make a comparison based on this unsupported claim.

Default assumption based on common knowledge Sometimes the reasoning model may default to assumptions based on general world knowledge or statistical priors. For example, consider the question: "*Who was born later, **Bradley Tyler Johnson** or **Bernardo Freitas**?*" In the absence of retrieved birthdate information for both individuals, the model responds by reasoning that many Brazilian athletes are typically born in the 1990s or later. Based on this assumption, it concludes that *Bernardo* is likely the younger individual. While this guess may seem plausible, it is not grounded in any retrieved facts.

Table 7: Results table for 2wikimultihop dataset using wiki2018 corpus

2WikiMultiHopQA					
Method	LLM	Retriever	Corpus	LLM Eval	Cover-EM
Search-o1	QwQ-32B	Cortex Search	wiki2018	0.664	0.622
Ours	Qwen2.5-72B-Instruct	Cortex Search	wiki2018	0.484	0.502
Ours	GPT-4o	Cortex Search	wiki2018	0.574	0.564

Table 8: Grounding issues in experiments

	Problem	Proportion
1	Assumption under insufficient information	6.0%
2	Misidentification under semantic similarity	3.0%
3	Factual hallucination without retrieval	0.6%
4	Default assumption based on common knowledge	0.4%
5	Random selection under uncertainty	0.2%
6	Attribution under insufficient information	0.2%
	Total	10.4%

Random selection under uncertainty In scenarios where the retrieved evidence does not provide sufficient information to support either option, the model may still attempt to produce an answer by making an arbitrary or speculative choice. For example, consider the question: *"Who is younger, **Jan Britstra** or **Leonas Milčius**?"* In this case, there is no retrieved information containing the birthdates or ages of either individual. Despite the lack of evidence, the model responds with: *"Since I have to choose, perhaps the intended answer is that **Leonas Milčius** is younger, so I'll go with that."* This behavior reflects an ungrounded decision made under uncertainty, where the model chooses an answer simply to avoid abstaining.

Attribution under insufficient information The reasoning model may attempt to attribute information by referencing partial or indirectly related mentions. For example, consider the question: *"Which film has the director born later, **Cose Da Pazzi** or **Kadhalil Sodhappuvadhu Yeppadi**?"* In this case, no reliable birth-year information is found for the actual director of *Cose Da Pazzi*. However, the model references possible candidates based on partial associations with the film or Italian cinema. It then infers that the director of the Italian film is likely older, and concludes that *Kadhalil Sodhappuvadhu Yeppadi* has the younger director. This attribution, while seemingly reasonable, is based on indirect evidence.

D LLM Prompts and Templates

In this section, we present the instruction prompts and the input/output template provided to LLMs.

Prompt for Question Decomposition - Example Part

Question: What is the capital of France?

Reasoning: The question asks for a direct fact that does not require multiple steps or dependencies.

Output: None

Question: Who is the CEO of the company owns the brand that produces iPhones?

Reasoning: To answer this question, it is necessary to identify the company associated with the brand producing iPhones first. Once the company is determined, its CEO can be identified. This logical dependency makes decomposition necessary.

Output:

1. What is the company owns the brand that produces iPhones?

2. Who is the CEO of #1?

Question: Which country has a larger population, Canada or Australia?

Reasoning: The question requires a comparative analysis. To perform the comparison, the populations of Canada and Australia must first be individually achieved. Then, the comparison can be made based on the given values.

Output:

1. What is the population of Canada?

2. What is the population of Australia?

3. Canada has a population of #1. Australia has a population of #2. Which country has a larger population, Canada or Australia?

Question: Which river is longer, Nile or Tigris?

Reasoning: This question requires determining the lengths of the two rivers to make a comparison. The lengths of each river must first be individually identified before a conclusion can be drawn.

Output:

1. What is the length of Nile?

2. What is the length of Tigris?

3. The length of Nile is #1. The length of Tigris is #2. Which river is longer, Nile or Tigris?

Question: What is the capital city of the country where the painter of Starry Night was born?

Reasoning: The question involves a chain of relationships. First, the painter of Starry Night must be identified. Then, the country where this painter was born needs to be determined. Finally, the capital city of that country must be retrieved. Each step depends on the outcome of the previous one, making decomposition necessary.

Output:

1. Who is the painter of Starry Night?

2. What is the country where #1 was born?

3. What is the capital city of #2?

Question: What is the distance between the tallest mountain in Japan and the tallest mountain in Nepal?

Reasoning: To find the distance, it is first essential to identify the tallest mountains in Japan and Nepal individually. Once both are identified, their geographical distance can be obtained. Decomposition ensures clarity and systematic problem-solving.

Output:

1. What is the tallest mountain in Japan?

2. What is the tallest mountain in Nepal?

3. What is the distance between #1 and #2?

Question: What is the distance between the city where the Colosseum is located and the capital city of the country where the Eiffel Tower is located?

Reasoning: This question involves two distinct entities and requires their geographical locations to be identified. First, the country where the Eiffel Tower is located must be found, followed by identifying its capital city. Next, the city where the Colosseum is located must be identified. Finally, the distance between these two locations can be determined. The multiple layers of reasoning and dependencies make decomposition necessary.

Output:

1. What is the country where the Eiffel Tower is located?

2. What is the capital city of #1?

3. What is the city where the Colosseum is located?

4. What is the distance between #3 and #2?

Prompt and Template for Question Decomposition

You are an expert skilled at analyzing complex questions and decomposing them into distinct and simpler sub-questions.

Your task is to analyze the input question and determine whether it requires decomposition.

Please give the reasoning that explains why decomposition was needed and the logic behind breaking the question into sub-questions.

If the question is already simple and does not require decomposition, output "None". Otherwise, decompose the question into multiple distinct, interconnected, self-contained sub-questions.

If a sub-question depends on the answer to a previous sub-question, refer to the previous sub-question using "#number", don't use the "based on".

For questions that are simple, they do not require identifying multiple intermediate steps or facts before answering. They can be directly answered by retrieving a single piece of information or directly comparing two known entities without additional intermediary steps.

For questions that require multiple steps of reasoning or fact-finding, where each sub-question logically depends on the answer to a previous one. For these, the sub-questions should be interconnected such that once all of them are answered in sequence, the original question can be answered.

For questions that need to do comparison, the final sub-question should present the information from previous sub-questions using declarative sentences and reiterate the original question so that the comparison can be made based on the gathered information.

Input fields are:

Question: {the input question}

Output fields are:

Reasoning: {reasoning for how to decompose the question}

Output: {the decomposed sub-questions}

Prompt and Template for Question Construction

You are given the original question, a list of answered sub-questions and their answers based on the original question.

For each question-answer pair, the question is labeled with a number as "Question #n", and each corresponding answer is labeled with the same number as "Answer #n", we also have reasonings for getting each answer, which is labeled as "Reasoning #n".

You then have a new question that may reference earlier answers using these labels "#n".

Your task is to rewrite this new question by replacing every reference (like "#1") with the corresponding answer from the list of previous questions and answers.

To output the good rewritten question, you should:

- Identify all placeholders in the new question (such as "#1" "#2" etc.).
- Find the corresponding answer from the list of previous question-answer pairs.
- From that answer, extract the key information that logically replaces the placeholder, ignore any additional background information or explanations. This key information might be a single word or a phrase that best fits the context of the new question.
- Rewrite the new question, replacing each placeholder with the key information extracted from the corresponding answer.
- If the corresponding answer did not provide any useful information, please based on the original question to rewrite this sub-question, make it understandable and answerable.
- If necessary, adjust the wording so the output rewritten question is grammatically correct and logically coherent.
- If there are no placeholders or no relevant information to substitute, return the original new question unchanged.

Input fields are:

Original Question: {the original question, following sub-questions are derived from this question}

Question-Answer Pairs: {the previous questions and answers}

New Question: {the new question that needs to be rewritten}

Output fields are:

Rewritten Question: {the rewritten output}

Prompt and Template for Retrieval Decision

You are given a new question. Your task is to determine whether the question already contains enough information to be answered without retrieving additional information.

If the question includes all the necessary details, output "false" (no retrieval needed).

If the question is missing information and cannot be answered as-is, output "true" (retrieval needed).

To output the correct decision, you should:

- a. Check the question and analyze it thoroughly.
- b. If everything required to answer it is explicitly stated or can be inferred from the text of the question itself, respond with "false" (meaning no retrieval needed).
- c. Otherwise, if key information is not provided, respond with "true" (meaning retrieval is needed).
- d. DO NOT rely on your own knowledge to answer the question. Make your decision based solely on the information provided in the question text.

For example:

Question: What is the birth date of Lily?

Analysis: The question does not provide a birth date, so we need additional info.

Output: true

Question: Alice was born on April 6, 2001 and Jack was born on May 15, 2002. Which person was born earlier?

Analysis: The question includes all needed information in the text. We just need to do a comparison to get the answer.

Output: false

Input fields are:

Question: {the input question}

Output fields are:

Analysis: {the analysis for whether needs retrieval}

Output: {true or false}

Prompt and Template for Query Rewriting

You are an expert in refining and optimizing retrieval queries.

Your task is to rewrite the current question into a concise and effective retrieval query, ensuring it captures the key intent and is optimized for precision.

Additionally, if "Last Rewritten Query" is provided, use it as a reference for continuity and improvement.

If "Last Rewritten Query" is "none", assume the original question was used as-is for retrieval and rewrite the query from scratch.

For example:

Example 1:

Question: What is the capital of France?

Last Rewritten Query: Capital city of France

New Query: France capital city

Example 2:

Question: What is the tallest mountain in the world?

Last Rewritten Query: none

New Query: Tallest mountain in the world

Input fields are:

Question: {the input question}

Last Rewritten Query: {last time's rewritten query}

Output fields are:

New Query: {the new rewritten query}

Prompt and Template for Passage Reranking

You are an intelligent assistant that can rank passages based on their relevancy to the query. We will provide you with a few passages, each indicated by a numerical identifier []. Rank the passages in how well their content can be used to directly answer the query. Only output the top-10 most relevant passages, listed in descending order of relevance. If fewer than 10 passages are provided, rank all available passages. Please give the reasoning for ranking the passages. The final output format should be [x] > [y] > [z] > [w] > [v] and should contain exactly 10 numerical identifiers if 10 or more are available, or all available identifiers if fewer than 10 are provided. You can only use ">", using "=" and "<" is prohibited. This means you should always give descending ranking, no equal and ascending ranking. For example:
Identifiers: [1], [2], [3], [4], [5]
Output: [2] > [3] > [5] > [1] > [4]
Identifiers: [1], [2], [3], [4], [5], [6], [7], [8], [9], [10]
Output: [2] > [9] > [5] > [8] > [4] > [1] > [10] > [7] > [3] > [6]
Only respond with the ranking results, do not say any word or explain.

Input fields are:
Query: {the input search query}
Identifiers: {all the numerical identifiers of given passages}
Context: {the passages need to be selected and ranked}
Output fields are:
Reasoning: {reasoning for how to select and rank the given passages}
Output: {the ranking results}

Prompt and Template for Answer Generation

Answer the question using only the information in the provided context. You should reason through the context to get the answer. Clearly present the relevant information in your reasoning. If, after careful reasoning, the context does not provide enough information to answer the question, respond with: "I don't know." The question is a simplified step (one hop) derived from a more complex question. To help remove ambiguity, we may provide background information including the original full question and previous answers to previous related sub-questions. Use this background only to understand the intent of the question—do not extract facts or cite anything from it. Background may be empty in some cases. You MUST include a citation for **every** factual statement in your answer using the context number(s) in square brackets immediately after the relevant information. Only omit citations if the context does not support any part of the answer and you respond with: "I don't know." Citation format: [number], or [number1][number2] if supported by multiple contexts. Examples:
Answer: He went to school at 8 am [1].
Answer: She wrote book A [2] and book B [4].
Answer: They are shopping for food [1][3].
Your answer must be accurate, direct, complete, and concise. Do not include additional background or explanation beyond what is necessary to answer the question.

Input fields are:
Question: {the question needs to be answered}
Context: {may contain relevant information}
Background: {the original question and other question hops with answers}
Output fields are:
Reasoning: {reasoning to output the answer}
Output: {the output answer}

Prompt and Template for Answer Verification

You are a verifier. Your task is to determine whether the answer correctly solves the question and is well-supported by the cited context(s).

You are given a question, an answer, and a set of citations. Follow this two-step process:

Step 1: Answer Quality

Determine whether the answer correctly, directly, and fully answers the question.

It must not be vague, evasive, or incomplete.

If the answer fails to provide a valid, specific response to the question: for example, by saying "not enough information" or "I don't know", you must output "false".

Step 2: Citation Support (only if Step 1 passes)

Check that if every claim in the answer is explicitly supported by the cited context(s).

The answer must not include information that is missing or unsupported by the citations.

Always provide a brief explanation in the reason section.

Please output your final judgment as either "true" (the answer is correct and fully supported) or "false" (the answer is incorrect or not supported).

Input fields are:

Question: {the input question}

Answer: {the given answer}

Source: {the associated citations}

Output fields are:

Reason: {reason of choosing true or false}

Output: {true or false}

Prompt and Template for Final Answering

A multi-hop reasoning question is one where the answer cannot be found by looking at a single piece of information or taking just one step.

Instead, it requires combining information from multiple sources or completing several logical steps.

To tackle such questions, we need to decompose the main question into smaller, more manageable sub-questions.

By answering these sub-questions one by one, you build up to the final answer.

Now we will provide a multi-hop reasoning question, with a set of its decomposed sub-questions.

We also provide the reasonings and answers to each sub-questions.

Your task is to combine these pieces of information to answer the original main question thoroughly and accurately.

Please articulate your reasoning for the final answer.

To achieve the good output, you should:

- Read the sub-questions with their reasonings and answers carefully.
- Integrate or synthesize the answers to the sub-questions into a single, coherent answer to the original question.
- Ensure your final answer is simple, clear, concise, and correct.

Input fields are:

Question: {the original multi-hop reasoning question}

Decomposed Information: {the answered sub-questions}

Output fields are:

Reasoning: {reasoning to output the answer}

Answer: {the output answer}

Prompt and Template for Improve Analysis in Self-Reflection

You are an expert in multi-hop question answering and decomposition.

Your task is to analyze a provided question decomposition and offer high-level guidance on how to correct and improve it, if necessary.

Use the rewritten sub-questions, reasoning, and answers only to detect decomposition flaws—not to inform your revised logic.

You will receive:

- a. A complex question requiring multi-hop reasoning.
- b. A record of sub-questions, including: the original decomposition, rewritten sub-questions, reasoning steps, and answers.

Assume the current decomposition may be incorrect. Provide structured, bullet-pointed instructions on how to improve it by:

- a. Proposing a logical, step-by-step breakdown of the question.
- b. Ensuring sub-questions extract critical information in the right order.
- c. Avoiding dependency errors or propagation of uncertain information.

If the decomposition is already sound and each sub-question is clearly answerable, respond with:

The previous decomposition is appropriate. Please follow the same structure.

Do not provide new sub-questions. Please only give high-level instructions on how the question should be decomposed.

Input fields are:

Question: {the input question}

Sub-questions Solving Record: {the previous decomposed sub-questions and their solving process}

Output fields are:

Analysis: {instructions for the new decomposition logic}

Prompt and Template for Improve Decomposition in Self-Reflection

You are an expert in multi-hop reasoning and question decomposition.

Your task is to analyze the original question and the previous one or multiple decompositions, which have been determined to be incorrect, and generate a new, logically sound decomposition that leads to the correct final answer.

Important instructions:

- The previous decomposition(s) are incorrect and led to wrong final answers.
- Do not rely on the structure of the previous decomposition(s) when generating a new one. Instead, construct a fresh, logically valid decomposition directly from the original question.
- If multiple incorrect decompositions are provided, analyze and identify the reasoning flaws in each before generating the new decomposition.
- You can use the instructions for help when constructing the new decomposition.
- Provide reasoning that explains the logic behind the new decomposition. The new decomposition logic should be different from the previous ones.
- Ensure that the new sub-questions follow a proper reasoning order and can be answered step-by-step to reach the final answer.
- If a sub-question depends on a previous answer, explicitly reference it using "#number" rather than vague phrasing like "based on".

For example:

Question: Which city is home to the headquarters of the company founded by the inventor of the telephone?

Previous decomposition:

Incorrect decomposition 1:

- Who invented the telephone?
- Which company was founded by #1?
- Which city is home to the headquarters of #2?

New decomposition instructions 1:

The second step is too vague. Instead of asking about where the company is located, specify that we need the ****headquarters**** location.

Reasoning: The previous decomposition jumps directly from identifying the company to identifying the city, skipping a crucial intermediate step. Before determining which city the headquarters is in, we should first establish the exact headquarters location of the company. This makes the reasoning more structured and avoids potential errors if a company has multiple locations.

New decomposition:

- Who invented the telephone?
- Which company was founded by #1?
- Where are the headquarters of #2?
- Which city is home to #3?

Input fields are:

Question: {the input question}

Previous Decompositions: {the previous decomposed sub-questions and the instructions for improving}

Output fields are:

Reasoning: {reasoning to the new decomposition}

New Decomposition: {the new decomposition based on the question}

Prompt and Template for Final Answer Verification

You are a verifier tasked with determining whether a given answer is correct and well-supported by the provided passages.

Given a question, an answer, and the supporting passages, evaluate the answer based on the following criteria:

- The answer must be factually correct based on the provided passages.
- The answer must fully and directly address the question without omitting critical details necessary for a comprehensive response.
- The answer must be explicitly substantiated by the provided passages, meaning that every claim made in the answer must be directly supported by the passages without requiring external knowledge.
- If the passages do not contain enough information to support the answer, or if the answer includes information not found in the passages, the answer should be considered incorrect.
- The question always has an answer. If no valid answer can be determined from the passages, it means either the question itself has problems or the necessary supporting passages are missing. In this case, the answer is incorrect by default. You **MUST** output false, no exceptions.

You need to provide a brief justification explaining why the answer is correct or incorrect based on the passages in the reason section.

Output true if the answer is factually accurate, relevant, complete, and fully supported by the passages. Otherwise, output false.

Input fields are:

Question: {the input question}

Answer: {the given answer}

Passages: {the supporting passages for the answer}

Output fields are:

Reason: {why the answer is correct or incorrect}

Output: {true or false}

Prompt and Template for LLM Evaluation

You are an evaluator responsible for determining whether a given answer correctly aligns in meaning with a provided ground truth answer.

Evaluation Criteria:

- Relevance:** The answer must directly address the given question and provide a coherent, contextually appropriate response.
- Semantic Alignment:** Compare the answer with the ground truth(s) to ensure they convey the same core meaning or intent, even if phrased differently.
- Completeness:** The answer must include all essential information present in the ground truth.

Ground Truth Format:

- The ground truth is provided as a list, containing one or multiple valid expressions of the correct answer.
- If multiple ground truths are given, they are semantically equivalent but differ in wording.

Output Instructions:

- Output true if the answer aligns in meaning with at least one of the ground truth expressions and includes all key details.
- Output false if the answer fails to align with any of the ground truths or omits important information.

Input fields are:

Question: {the input question}

Ground Truth Answer: {the ground truth answer}

Our Answer: {the answer needs to be evaluated}

Output fields are:

Reasoning: {reasoning to determine whether our answer is correct compared to the ground truth}

Output: {true or false}